INHERITANCE, ITERATORS, & GENERATORS Meta

COMPUTER SCIENCE MENTORS 61A

October 19, 2026 – October 23, 2026

Example Timeline

- **This is a menu of problems, not a requirement to finish the worksheet.** Individual suggested times do not add up to one hour. Choose or skip problems based on your students, and give each mini-lecture immediately before its section.
- Check-in and icebreaker [5 min]. Ask students for an example of an "is-a" relationship (for instance, a checking account is an account).
- Inheritance: Q1 (WWPD with `Elphaba` and `Glinda`), then Q2 (`ForceWielder`) or Q3 (`TeamBaller`) [20 min]. Q1 builds the lookup chain and **`type`**`(self)`; Q2 adds **`super`**`()` and class attributes; Q3 practices overriding and calling the base class's method.
- Iterators: Q4 (iterator versus iterable), then Q5 and Q6 (WWPD) [10 min]. Q7 is a challenge: students work backward from the printed output by tracking iterator positions.
- Generators: Q8 (WWPD) [8 min], then Q9 (`accumulate`) for generator syntax with an infinite iterator [7 min].
- Challenge: Q10 (`fruitOptions`) is a recursive generator. Use it if students are comfortable with **`yield`** in a **`for`** loop over a recursive call.
- Wrap-up and questions [5 min]. Ask students to explain the difference between calling a function and calling a generator function.

Teaching Goals

- A subclass inherits from a base class without copying its attributes. Attribute lookup checks the instance, then the class, then each base class up the chain, and only ever goes up.
- Overriding replaces an inherited method in the subclass; **`super`**`()` (or `Base.method(self, ...)`) lets the new method call the old one, and `self` remains the subclass instance.
- Lookup and assignment differ: assignment through an object writes to that object, so `self.x = ...` makes an instance attribute and `Class.x = ...` changes the class attribute.
- **`iter`** creates an iterator from an iterable, and **`next`** advances it. Two names for one iterator share a position; two calls to **`iter`** create two iterators.
- **`map`**, **`filter`**, **`zip`**, and generators are lazy: nothing is computed until a consumer asks for the next element. **`list`** comprehensions are eager.
- A generator function returns a generator without running its body. The body runs on **`next`**, pauses at **`yield`**, and resumes where it left off; the body finishing raises `StopIteration`.
- In recursive generators, use **`yield from`** (or a **`for`** loop) to pass along the values from a recursive call, and **`yield`** to produce your own values.

Created by Cadence Loh, Ana Abell, Tien-Yu Hsu, and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

1 Inheritance

1. What would Python display in the land of Oz?

```
class Elphaba(object):
    magic = 'Defying Gravity'

    def __init__(self, magic):
        self.magic = magic

    def spell(self):
        return type(self).magic + ' and ' + self.magic

class Glinda(Elphaba):
    magic = 'Popular'

    def __init__(self, magic):
        Elphaba.__init__(self, 'good ' + magic)

elphaba = Elphaba('Wicked')
```

(a)
```
>>> Elphaba.magic
'Defying Gravity'
```

(b)
```
>>> elphaba.magic
'Wicked'
```

(c)
```
>>> elphaba.spell()
'Defying Gravity and Wicked'
```

(d)
```
>>> Elphaba.spell()
Error
```

(e)
```
>>> Elphaba.spell(elphaba)
'Defying Gravity and Wicked'
```

(f)
```
>>> glinda = Glinda('Power')
>>> Glinda.magic

'Popular'
```

(g)
```
>>> glinda.magic
'good Power'
```

(h)
```
>>> glinda.spell()
'Popular and good Power'
```

2. What would Python display? Write the result of executing the following code and prompts. If nothing would happen, write "Nothing". If an error occurs, write "Error".

```
class ForceWielder():
    force = 25

    def __init__(self, name):
        self.name = name

    def train(self, other):
        other.force += self.force / 5

    def __str__(self):
        return self.name

class Jedi(ForceWielder):
    lightsaber = "blue"

    def __str__(self):
        return "Jedi " + self.name

    def __repr__(self):
        return f"Jedi({repr(self.name)})"

class Sith(ForceWielder):
    lightsaber = "red"
    num_sith = 0

    def __init__(self, name):
        super().__init__(name)
        Sith.num_sith += 1
        if self.num_sith != 2:
            print("Two there should be. No more, no less.")

    def __str__(self):
        return "Darth " + self.name

    def __repr__(self):
        return f"Sith({repr(self.name)})"
```

```
>>> anakin = Jedi("Anakin")
>>> anakin.lightsaber, anakin.force
```

```
("blue", 25)
```

```
>>> obiwan = Jedi("Obi-wan")
>>> anakin.master = obiwan
>>> anakin.master
```

```
Jedi("Obi-wan")
```

```
>>> Jedi.master
```

```
AttributeError
```

```
>>> obiwan.force += anakin.force
>>> obiwan.force, anakin.force
```

```
(50, 25)
```

```
>>> obiwan.train(anakin)
>>> obiwan.force, anakin.force
```

```
(50, 35.0)
```

```
>>> Jedi.train(obiwan, anakin)
>>> obiwan.force, anakin.force
```

```
(50, 45.0)
```

```
>>> sidious = Sith("Sidious")
```

```
Two there should be. No more, no less.
```

```
>>> ForceWielder.train(sidious, anakin)
>>> anakin.lightsaber = "red"
>>> anakin.lightsaber, anakin.force
```

```
("red", 50.0)
```

```
>>> Jedi.lightsaber
```

```
"blue"

>>> print(Sith("Vader"), Sith("Maul").num_sith)


Two there should be. No more, no less.
Darth Vader 3

>>> rey = ForceWielder("Rey")
>>> rey


<__main__.ForceWielder object>

>>> rey.lightsaber


AttributeError
```

To address a point brought up in previous semesters: You may realize that as both a ForceWielder and Jedi, `anakin` has no attribute named `master` at first. Note to students that it is possible in our OOP objects for us to declare object attributes on the fly. Also communicate to students that while you can declare object attributes on the fly, that does not mean that such attributes persist to the class the object exists in as a whole.
In my opinion, going through an example like this is far more helpful for students than a mini-lecture. Try to foresee some questions and confusions might have and how you might address them, for example:

- Why, in the `__init__` method of `Sith` can we use `self.num_sith` instead of `Sith.num_sith`? And why can't we write `self.num_sith += 1`?
- Why does evaluating `rey` give us `<__main__.ForceWielder object>`, but this is not the case when we evaluated `anakin.master`?
- What's going on with `ForceWielder.train(sidious, anakin)`?
- Can we write `Jedi.train(sidious, rey)`, even though neither `rey` nor `sidious` are Jedi?

These are also questions you could bring up if students don't ask them.

3. Write `TeamBaller`, a subclass of `Baller`. An instance of `TeamBaller` cheers on the team every time it passes a ball/states it doesn't have the ball when it doesn't and returns a boolean of whether or not it passed the ball. If the `TeamBaller` did not pass the ball, it'll say it hasn't done so and return False. Assume `Baller` has a defined *pass_ball* method.

```
class Baller:
    all_players = []
    def __init__(self, name, has_ball = False):
        self.name = name
        self.has_ball = has_ball
        Baller.all_players.append(self)

    def pass_ball(self, other_player):
        if self.has_ball:
            self.has_ball = False
            other_player.has_ball = True
            return True
        else:
            return False

class BallHog(Baller):
    def pass_ball(self, other_player):
        return False


class TeamBaller(Baller):
    """
    >>> alyssa = BallHog('Alyssa')
    >>> cheerballer = TeamBaller('Esther', has_ball=True)
    >>> cheerballer.pass_ball(alyssa)
    Yay!
    True
    >>> cheerballer.pass_ball(alyssa)
    I don't have the ball
    False
    """
    def pass_ball(self, other):
        did_pass = Baller.pass_ball(self, other)
        if did_pass:
            print('Yay!')
        else:
            print("I don't have the ball")
        return did_pass
```

Teaching Tips

- This is mean to be an introduction to how object interact with other objects, overwriting methods, and inheritance. As of the Spring 2024 iteration of this course, students are fairly new to the concept of inheritance, and as such, may need a little hand-holding through this problem. Let this serve as a gentle introduction and if they get confused on nuances, clarify.
- Let students think about what methods are being inherited, along with the attributes that are inherited in the init method.
- Remember that although the pass_ball method will be overridden in the TeamBaller method, it can call the inherited class's method.
- Remember that the pass_ball method of the Baller class will return whether the ball was passed or not, which decides whether the TeamBaller instance should cheer or not.

2 Iterators

An **iterable** is any object whose elements can be processed one at a time, such as a list, string, dictionary, or `range`. An **iterator** is an object that keeps track of a position in a sequence and can produce the next element on request.

- `iter(iterable)` returns a *new* iterator over the elements of the iterable. Calling `iter` on an iterator returns that same iterator.
- `next(iterator)` returns the next element and advances the iterator. When no elements remain, it raises `StopIteration`.

```
>>> s = [3, 4, 5]
>>> t = iter(s)
>>> next(t)
3
>>> next(t)
4
>>> u = iter(s)         # a separate iterator with its own position
>>> next(u)
3
>>> w = t               # not a new iterator: another name for the same one
>>> next(w)
5
>>> next(t)
StopIteration
```

Iterators only move forward, and calling `next` on any name for an iterator advances it for every name. A `for` loop calls `iter` on its iterable and repeatedly calls `next` until `StopIteration`.

Many built-in functions return iterators that compute their results **lazily**, meaning each element is computed only when `next` asks for it:

- `map(func, iterable)`: iterate over `func(x)` for each `x` in the iterable
- `filter(func, iterable)`: iterate over each `x` in the iterable for which `func(x)` is true
- `zip(first_iter, second_iter)`: iterate over pairs `(x, y)`
- `reversed(sequence)`: iterate over the sequence in reverse

Functions such as `list`, `sum`, `min`, `max`, `all`, and `any` consume an iterable. `all` stops at the first false value and `any` stops at the first true value, so they may not use up the whole iterator.

The most common confusion is thinking `d = c` makes a copy of an iterator. Draw one iterator object with two arrows to it, and one list with an arrow from the iterator showing its position. Contrast with `iter(a)` called twice, which creates two iterator objects pointing at the same list.

When introducing laziness, ask students what a list comprehension prints versus `map` when the function has a `print` side effect. The `map` object does nothing until something consumes it. Also remind students that a list iterator looks at the *current* list, so appending to the list before the iterator is exhausted affects what it produces.

4. What is difference between an iterator and an iterable?

   Iterator: Mutable object that tracks a position in a sequence, advancing on each call to **`next`**

   Iterable: Represents a sequence and returns a new iterator on each call to **`iter`**

   To use in an English sentence: Lists are "iterable". To go through a list, you make an object called an "iterator" to scan through the list.

5. What Would Python Display? If you think nothing will be displayed, write N/A.

```
>>> a = [1, 2, 3]
>>> x = iter(a)
```

```
N/A
```

```
>>> next(x)
```

```
1
```

```
>>> next(x)
```

```
2
```

```
>>> y = iter(a)
>>> next(y)
```

```
1
```

```
>>> next(x)
```

```
3
```

```
>>> next(x)
```

```
StopIteration Error
```

```
>>> z = iter(y)
>>> next(z)
```

```
2
```

```
>>> [next(y), next(y), next(z)]
```

```
[2, 3, 3]
```

```
>>> a = iter(filter(lambda x: x % 2, map(lambda x: x - 1, range(10))))
>>> next(a)
```

```
-1
```

```
>>> reduce(lambda x, y: x + y, a)
```

```
16
```

6. WWPD? Write what this python program will print.

```
a = [1, 'A', 'B', 'C', 5, 6, 7, 'D', 'E']
x = iter(a)
for i in range(5 - next(x)):
    next(x)
print(next(x))
y = iter(a)
print(next(y))
z = iter(y)
print(next(z))
```

```
6
1
A
```

Teaching Tips

- iter(y) returns z so z is the same iterator as y.
- Every call to next moves the iterator's pointer forward by one.

7. Find an input to the year function that prints the following output: 2 0 2 6

```
def year(a):
    x = iter(a)
    y = iter(a)
    z = iter(x)
    for i in range(next(x)):
        y = iter(a)
        next(y)
    print(next(x))
    print(next(z))
    print(next(y))
    print(next(z))
```

```
year([__, 2, 0, 6, _____])
There can be any positive number in the first blank, and there can be any
   number of values after the 6.
```

Teaching Tips

- The first step to this problem is drawing out the initial states of the x and y iterators as pointers under the element at index 0 of an unknown list
- Recognize that iter(x) returns the iterator x itself, so z is x.
- Note that the for loop header moves the x iterator's position up by one. The body of the for loop assigns y to a new iter(a) so no matter how many times the for loop runs, the for loop will always end with the y iterator pointing to the item at the first index of a.
- We want the function to print 2, 0, 2, and 6 in that order. We know that the x iterator is pointing to the element at index 1 of the list, so the element at index 1 must be a 2. Calling print(next(x)) should print 2 and move the x iterator to point to the element at index 2.
- We know that the z iterator is the same as the x iterator, so since it is pointing the the element at index 2, the element at index 2 must be 0. Calling print(next(z)) should print 2 and also moves the iterator up to point to the item at index 3 in the list.
- We know from the end of the for loop that the y iterator points to the item at index 1, which we want to be 2. Fortunately, we already know that this item is 2 from when we figured out what print(next(x)) was.
- Lastly, we look at the z iterator (the same as the x iterator) and see its position is under the item at index 3, so we know that the item at index 3 should be 6.

3 Generators

A **generator function** is a function whose body contains **`yield`**. A normal function returns once; a generator function can yield many times. A **generator** is an iterator created automatically by calling a generator function.

```
>>> def plus_minus(x):
...     yield x
...     yield -x
>>> t = plus_minus(3)     # no body code runs yet
>>> t
<generator object plus_minus ...>
>>> next(t)               # run until the first yield
3
>>> next(t)               # resume after the first yield
-3
>>> next(t)               # the body finished
StopIteration
```

Calling a generator function does not run its body. The body runs only when **`next`** is called, and it pauses at each **`yield`**, keeping its local variables and its place in the code until the next call to **`next`**. When the body finishes (or reaches **`return`**), `StopIteration` is raised. Each call to a generator function creates an independent generator.

`yield from` `<iterable>` yields each element of the iterable in order, and is shorthand for a loop:

```
def a():
    yield from range(3)
```

```
def b():
    for x in range(3):
        yield x
```

Recursive generators combine **`yield from`** (to pass on everything the recursive call produces) with ordinary **`yield`** (to add values of their own).

Emphasize the difference between executing a function and executing a generator function. The call `plus_minus(3)` runs none of the body. Draw the generator as a box storing `x = 3` and a bookmark at the current **`yield`**; each **`next`** moves the bookmark forward.

Remind students that a **`for`** loop over a generator stops silently on `StopIteration`, and that a generator that has already been advanced starts from where it left off, not from the beginning.

8. Given the following code block, what is output by the lines that follow?

```
def foo():
    a = 0
    if a == 0:
        print("Hello")
        yield a
        print("World")

>>> foo()


<generator object>


>>> foo_gen = foo()
>>> next(foo_gen)


Hello
0


>>> next(foo_gen)


World
StopIteration


>>> for i in foo():
...   print(i)


Hello
0
World
```

Teaching Tips

- Emphasize heavily the fact that when generators are called, they return a generator object. They do NOT start executing their function body until after `next` is called! (So what does that first line return? A generator object!)
- Remind students that generator objects are independent from one another; if you create a new one from calling the same function again, it starts from the beginning again. Each generator on its own, however, remembers where it stopped after the previous `next` call, so that it can resume the next time you call `next`.
- What happens when there are no more `yield` statements, like in the second `call` on `foo_gen`? The generator has reached the end of all possible values to iterate over, and so it returns a StopIteration error.
- If you stick a generator object inside a for loop (or a list, for that matter), it will go all the way through from start to finish, outputting each yield value after another.
  - Careful, however: 'start' doesn't necessarily mean the very first lines of the function or the first yield call; if you feed in a generator on which you've already called `next`, its "start" will be where it last left off.

9. Write a generator function that takes in an iterator `it` and yields the running total of the elements produced by `it`.

```
def accumulate(it):
    """
    >>> def all_ints():
    ...     i = 0
    ...     while True:
    ...         yield i
    ...         i += 1
    >>> a = accumulate(all_ints())
    >>> [next(a) for x in range(6)]
    [0, 1, 3, 6, 10, 15]
    """
```

```
def accumulate(it):
    total = 0
    while True:
        total += next(it)
        yield total
```

This problem is meant to be a relatively simple introduction to the manipulation of iterators, generator function syntax, and how to deal with infinite generators.

The doc test for this question is deliberately a bit complicated. It actually serves as a bit of a hint for how we might construct an infinite generator if one is provided to us (using an infinite loop). If students are concerned about this possibly erroring, remind them that generators only execute the body of the function when we tell them to, so this will not actually cause an infinite loop that hogs all of our computer's resources, causing a crash.

Students have a tendency to want to do everything in generators with lists, which requires the computation of all elements of the generator at the same time; we need to encourage them to do "lazy" iteration, which means that each element is only computed when it's needed. Lazy iteration is a core

concept of the topic; not only is it more efficient to do things this way, it's literally the only way to handle infinite iterators (whose elements cannot all be computed at the same time). Whenever I do iteration problems with my students, I ask them to consider what would happen if we provided them an infinite generator; if their implementation breaks, then it's not correct.

10. Write a generator that, given m (the amount of money you have), pc (the cost of one pear), and ac (the cost of one apple) yields all possible combinations of fruit that you can buy that uses up the most of your money. In other words, each combination of fruits should not result in enough money left over to buy another fruit. Combinations of the same number of each fruit in different orders is okay. It is also okay if each combination has an extra space at the end.

```
def fruitOptions(m, pc, ac):
    """
    >>> print(list(fruitOptions(10, 2, 5)))
    ['pear pear pear pear pear ', 'pear pear apple ', 'pear apple pear
       ',
     'apple pear pear ', 'apple apple ']
    """
    if ____________________________________:
        yield ""
    if m >= pc:
        for ____________________________________________:
            ____________________________________
    if m >= ac:
        for ____________________________________________:
            ____________________________________________


    def fruitOptions(m, pc, ac):
        if m < pc and m < ac:
            yield ""
        if m >= pc:
            for p in fruitOptions(m-pc, pc, ac):
                yield "pear " + p;
        if m >= ac:
            for a in fruitOptions(m-ac, pc, ac):
                yield "apple " + a;
```

Teaching Tips

- This problem shows that some generator problems can be approached in a similar way as recursion problems, except with yield instead of return.
- Another goal of this problem is to show how iterating through a generator can be done through a through a *for*__*in*___ loop.